PuDyDG at AISA-ArabicFC: Decomposed Fine-Tuning with Function-Centric Few-Shot Augmentation for Arabic Function Calling

Duy Dang Phu and Dang Van Thin
University of Information Technology, VNU-HCM, Ho Chi Minh City, Vietnam
Vietnam National University Ho Chi Minh City, Vietnam
24520010@gm.uit.edu.vn, thindv@uit.edu.vn

Abstract

Tool calling has become increasingly important for agentic systems, but remains challenging in multilingual settings, particularly for Arabic. We present our system for the AISA-ArabicFC shared task at ArabicNLP 2026. Our approach combines decomposed multi-task fine-tuning, function-centric few-shot augmentation, and hybrid schema-aware inference. On the development set, our system achieves a function-name accuracy of 1.000 and a normalized argument exact match of 0.858. On the blind test set, the released baseline re-scored with the final scorer obtains a Track A score of 0.8000, compared with 0.9111 for our submission. Our system achieves a function-name accuracy of 0.9973 and an argument exact match score of 0.8536 on this set, ranking seventh in Track A. Our solution is released in the GitHub repository[1].

1 Introduction

The rapid development of agentic systems has made tool use an important capability of large language models (LLMs). To interact reliably with external APIs, these models must identify the appropriate function and generate valid arguments from a user's request.

Despite recent progress in tool calling, existing evaluations remain predominantly English-centric, and recent work has only begun to systematically examine robustness under multilingual user interactions (Luo et al., 2026). Even when a model correctly understands the user's intent and selects the appropriate tool, its call may remain operationally invalid because the generated arguments violate language-invariant interface conventions. These findings motivate further study of Arabic tool calling, where systems must process linguistically diverse user requests while producing outputs that conform to rigid function schemas.

In this paper, we describe our system for the AISA-ArabicFC shared task at ArabicNLP 2026 (Nacar et al., 2026a). Starting from the task-provided `AISA-AR-FunctionCall-Think` model, we apply fine-tuning with a decomposition-based strategy. Although the system also generates the auxiliary reasoning trace evaluated in Track B, our system design and analysis primarily focus on the function-call fields evaluated in Track A. Each official training instance is reformulated into complementary objectives for tool-use decision, function-name prediction, argument extraction, function-call construction, and end-to-end response generation. At inference time, our hybrid pipeline combines function selection with argument extraction conditioned on the predicted function, followed by schema-aware post-processing. We further augment the argument-extraction objective with synthetic, function-centric Arabic requests generated from few-shot examples and the released function schemas. Our results suggest that task decomposition and targeted data augmentation are effective for improving Arabic function-calling performance, particularly argument extraction. Under the final scorer, the released baseline checkpoint obtains FNACC 0.9813, ARGEM 0.6790, and a Track A score of 0.8000 on the blind test set, compared with 0.9973, 0.8536, and 0.9111, respectively, for our submission. On the official test leaderboard, our submission under the team name *PuDyDG* ranked seventh with a Track A score of 0.9111, FNACC of 0.9973, and ARGEM of 0.8536 in Track A; the same submission ranked sixth in Track B.

2 Background

2.1 Problem Definition

Each AISA-ArabicFC instance presents an Arabic request, temporal context, and four candidate tools, each specified by a function name, description, and JSON parameter schema (Nacar et al.,

[1] https://github.com/phud774/PuDyDG-AISA-ArabicNLP

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1574–1579
October 28-29, 2026 ©2026 Association for Computational Linguistics

2026a). The Track A output is a pair (f, a): a candidate function name and a schema-compliant JSON argument object. For example, a request meaning "convert 5,000 Egyptian pounds to Kuwaiti dinars" maps to `convert_currency` with `amount=5000`, `from_currency=EGP`, and `to_currency=KWD`; a no-call request maps to $(\texttt{none}, \{\})$. Track A combines exact function-name accuracy (FNACC), including `none`, and argument exact match (ARGEM) after official value normalization as $0.40\,\text{FNACC} + 0.60\,\text{ARGEM}$. Tracks B and C additionally assess reasoning-trace generation and cross-dialect robustness, respectively.

2.2 Data and Evaluation

The official release contains 10,550 training instances (10,500 call/50 no-call), 545 development instances (500/45), and 1,125 blind test requests. Its task-oriented assistant queries span Modern Standard, Gulf, Egyptian, Levantine, and Maghrebi Arabic and eight service domains: healthcare, finance, government, Islamic services, travel, weather, e-commerce, and utilities. The registry contains 27 tools, including 20 target tools and seven distractors (Nacar et al., 2026a).

We use the full training split for adaptation and the development split only for validation. Synthetic examples use only positive training instances and released function schemas; development and test requests are excluded. We select 500 of 1,500 validated generations with seed 42. On development and the blind test set, we report FNACC, normalized ARGEM, and their official weighted score.

2.3 Related Work

Prior work has studied tool-use training and evaluation at increasing scales. Toolformer learns to invoke external APIs from demonstrations (Schick et al., 2023), while API-Bank and ToolLLM extend tool-use evaluation and instruction tuning to broader collections of APIs (Li et al., 2023; Qin et al., 2024). BFCL further evaluates function selection, argument construction, abstention, and multi-turn tool use (Patil et al., 2025). However, multilingual evaluations reveal substantial performance disparities across languages (Kulkarni et al., 2025). Models may select the correct function while still producing invalid arguments because user-language expressions do not conform to schema-defined fields and value conventions (Luo et al., 2026).

Recent work has begun to address Arabic tool calling. Arabic and bilingual instruction tuning improve tool-use decisions and argument generation, although cross-lingual transfer alone remains insufficient (Ersoy et al., 2025). Arabic adaptations of BFCL also report lower accuracy for Arabic queries, particularly in complex calling settings (Kubrak et al., 2025). Within the AISA framework (Nacar et al., 2026c), dataset auditing, schema repair, tool-aware prompting, and dialect-diverse fine-tuning have been shown to reduce structural errors (Nacar et al., 2026b). Our system complements these approaches by decomposing function calling into specialized training objectives, conditioning argument extraction on the selected schema, and applying function-centric Arabic augmentation specifically to the argument-extraction objective.

3 System Overview

Our system LoRA-adapts one model with five training views covering tool-use decision, function selection, argument extraction, call construction, and end-to-end generation. We add 500 function-centric examples for argument extraction and oversample no-tool cases. At inference, function selection and schema-conditioned extraction are combined with deterministic post-processing and a guarded end-to-end fallback.

3.1 Decomposed Multi-task Fine-tuning

Let an official training instance be $(x, \mathcal{F}, y)$, where x is the Arabic request, $\mathcal{F}$ contains four candidate schemas, and y is either a call (f, a) or a no-tool response. The *tool-use decision* view predicts a Boolean value; *function selection* predicts $f \in \mathcal{F}$ or `none`; and *argument extraction* predicts a JSON object conditioned on the gold function. For positive instances, *function-call construction* reproduces the structured call without `<think>`, whereas the *end-to-end* view retains the original reasoning span and call format.

A short Arabic instruction identifies each view. Candidate schemas, temporal context, and the user request are otherwise retained, except for argument extraction: we replace all candidates with a compact description of the gold schema, including types, required fields, and enumerated values. Its target contains only non-empty, evaluator-relevant arguments. Across views, the model is instructed to emit only values stated or directly inferable from the request and to omit missing values rather than invent defaults.

Examples are weighted through repetition before shuffling. Argument-extraction and end-to-end views are repeated twice, and every no-tool view receives an additional factor of 20. This prioritizes the more heavily scored argument objective while preserving a strong abstention signal.

3.2 Function-centric Few-shot Augmentation

Argument extraction must map varied Arabic expressions to rigid, language-invariant fields. We therefore group positive training instances by gold function and generate additional examples on a balanced function schedule. Each prompt contains five distinct same-function request–argument demonstrations, the compact schema, and the dialect and temporal context of an official template. The generator returns a new Arabic request and its JSON arguments.

We reject empty or duplicate examples, undeclared keys, missing required fields, invalid enumerated values, and values removed by the schema sanitizer; numeric and Boolean values are normalized when possible. Invalid candidates are retried with the validation error. Accepted examples are used only for argument extraction, avoiding synthetic supervision for selection, abstention, or reasoning traces.

3.3 Hybrid Schema-aware Inference

All inference passes use the same adapted model with greedy decoding. The selection pass emits one candidate function or `none`; only recognized names and no-tool markers are accepted. If parsing fails, a valid function from the independently generated end-to-end response is used, otherwise the system returns `none`. For Track B, the auxiliary reasoning trace is produced by the independently decoded end-to-end pass, which was trained on the original reasoning-span-and-call format. Its `<think>` span supplies the submitted trace when available; the final function name and arguments continue to be produced by the hybrid selection-and-extraction pipeline described below.

For $\hat{f} \neq \texttt{none}$, a second pass removes all candidate declarations and inserts only the schema of $\hat{f}$, then requests one JSON argument object. We parse the first balanced object as JSON, with a conservative literal parser for minor quoting errors. If parsing fails, the end-to-end arguments are used only when their function agrees with $\hat{f}$. For $\hat{f} = \texttt{none}$, extraction is skipped.

Finally, schema-aware post-processing removes undeclared, empty, and officially ignored fields, normalizes Arabic-Indic digits, casts numeric and Boolean values, and preserves identifier-like strings with leading zeros. The resulting $(\hat{f}, \hat{a})$ is the Track A prediction.

4 Experimental Setup

Baseline. We report the official development results originally published by the organizers for `AISA-AR-FunctionCall-Think 270M`, without retraining or reevaluating the checkpoint. The originally published development baseline used a pre-normalization evaluator and is therefore not used for direct comparison with our development results. For a like-for-like comparison, the organizers re-scored the same baseline checkpoint on the 1,125-instance blind test set with the released final scorer. It obtained FNACC 0.9813, ARGEM 0.6790, Track A 0.8000, and Track B 0.8074.

4.1 Training Configuration

We initialize the system from the task-provided `AISA-AR-FunctionCall-Think` checkpoint. LoRA is applied to all attention projections and the gate, up, and down MLP projections, with rank 64, scaling factor 128, and dropout 0.05.

Training runs for one epoch with a 2,048-token sequence limit. We use fused AdamW with a peak learning rate of 10^{-4}, cosine decay, 3% warm-up, and weight decay 0.01. A per-device batch size of 6 with eight gradient-accumulation steps gives an effective batch size of 48. Training uses BF16, gradient checkpointing, seed 42, and target-only loss.

4.2 Augmentation and Decoding Configuration

For each synthetic generation slot, we sample five distinct same-function demonstrations and query `Llama-3.3-70B-Instruct` at temperature 0.9. Invalid candidates are retried with the validation error, up to three attempts. At inference, maximum generation lengths are 24 tokens for selection, 160 for argument extraction, and 256 for end-to-end generation.

4.3 Reproducibility

The original training and inference run was performed on a temporary single-GPU Vast.ai instance. Seed 42 was fixed, but deterministic PyTorch/CUDA algorithms were not enforced, and

Table 1: Development-set single-run results. Batch configurations denote per-device batch size × gradient-accumulation steps.

Configuration	FnAcc	ArgEM
Decomposition		
Decomposed (4×8)	1.000	0.780
Training configuration		
+ separate argument model	1.000	0.744
Decomposed (6×8)	1.000	0.788
Decomposed (12×8)	1.000	0.786
Function-centric augmentation		
Decomposed (6×8; 0 augmented)	1.000	0.788
+ 250 augmented examples	1.000	0.844
+ 500 augmented examples (full)	1.000	**0.858**
+ 750 augmented examples	1.000	0.844
Training-only objective ablation		
− function-call construction view	1.000	0.856
− tool-use decision view	1.000	0.842

synthetic generation is stochastic; retraining or regenerating the data is therefore not expected to be byte-identical. We release the exact augmentation pool, adapter, tokenizer, configuration, and prediction artifacts. Inference from the released checkpoint on an NVIDIA RTX PRO 5000 reproduced the submitted prediction file byte for byte. Separately, the organizers independently evaluated the released reproduction run and obtained a Track A score of 0.9105, compared with the published submission score of 0.9111. This small difference is consistent with the disclosed sampling and training non-determinism.

5 Results

5.1 Development Results and Ablations

Among our single-run experiments in Table 1, the shared multi-task model reaches 0.788, compared with 0.744 for the separate argument model. The 6×8 configuration also gives the highest observed score among the tested batch settings, although its 0.002 difference from the 12×8 configuration corresponds to only one of the 500 positive development examples.

Adding 500 function-centric examples gives the highest observed ARGEM, 0.858, while both 250 and 750 examples obtain 0.844. These results are not monotonic and come from one run per configuration; the apparent peak at 500 examples may therefore lie within run-to-run variation rather than establish a reliable optimum.

The training-only objective ablations leave FNACC at 1.000 but yield ARGEM scores of 0.856 without function-call construction and 0.842 without tool-use decision. Relative to the full result of 0.858, these differences correspond to approximately one and eight positive development examples, respectively. They suggest beneficial multi-task transfer, but should not be interpreted as statistically robust without repeated runs. Neither objective adds an inference stage. All of our configurations in Table 1 are single training runs using seed 42, and no variance estimates are available.

5.2 Development-set Argument Error Analysis

We further inspect the released development predictions associated with an ARGEM of 0.844. This post-hoc analysis is distinct from the best development run (0.858) in Table 1. Among the 500 positive development instances, 78 have an argument mismatch under the official evaluator. In 55 cases (70.5%), the predicted and gold objects contain the same keys but differ in at least one value. Manual inspection reveals three main causes: span-boundary mismatches in free-text and list fields, semantic substitutions or hallucinated values, and a smaller number of ambiguous or inconsistent gold annotations. Thus, argument-value prediction, rather than field selection or output parsing, remains the main source of residual error.

Among well-represented dialects, mismatch rates range from 12.8% for MSA to 22.4% for Levantine, with Gulf (16.7%) and Egyptian (18.2%) between them. Maghrebi reaches 33.3%, but has only six positive examples. These figures suggest a dialect effect, although the smallest groups require cautious interpretation.

5.3 Official Test Results

Our submission under the team name *PuDyDG* ranked seventh in Track A, achieving a score of 0.9111, with FNACC of 0.9973 and ARGEM of 0.8536. Under the same released final scorer, the baseline checkpoint obtains FNACC 0.9813, ARGEM 0.6790, and Track A 0.8000 on the blind test set, providing a like-for-like comparison.

For Track B, which additionally evaluates the reasoning trace, our submission ranked sixth. The trace is supplied by the end-to-end generation pass trained on the original response format containing the `<think>` span. This pass is separate from the selection and schema-conditioned argument-extraction passes used to construct the final function call.

6 Limitations

Our study has four main limitations. First, each development configuration was trained only once with seed 42, and no variance estimates are available. Because the positive development subset contains only 500 instances, several observed differences correspond to only a handful of examples and may not be statistically robust.

Second, although 500 augmented examples produced the highest observed development ARGEM, the augmentation-size experiments were single runs. The difference between 500 and 750 examples may therefore reflect run-to-run variation rather than establish 500 examples as a reliable optimum.

Third, exact retraining reproducibility is limited. Synthetic examples were generated through stochastic sampling, and deterministic PyTorch/CUDA algorithms were not enforced. We release the exact augmentation pool, adapter, tokenizer, configuration, and predictions, which supports exact inference from the released checkpoint, but retraining or regenerating the augmentation data may produce different results.

Finally, our analysis primarily focuses on Track A. Although our submission ranked sixth in Track B and the reasoning trace is generated by the end-to-end pass, we do not provide a detailed quantitative or manual analysis of trace quality. Further work should examine the faithfulness, consistency, and contribution of these traces independently of function-call accuracy.

7 Conclusion

In this paper, we presented our system for the AISA-ArabicFC shared task at ArabicNLP 2026. The system combines decomposed multi-task fine-tuning, function-centric few-shot augmentation, and hybrid schema-aware inference. On the development set, our system achieved a normalized argument exact match of 0.858 while maintaining perfect function selection. On the blind test set under the final scorer, our system achieved Track A 0.9111 and ARGEM 0.8536, compared with 0.8000 and 0.6790, respectively, for the rescored baseline checkpoint. Our system ranked seventh in Track A and sixth in Track B, where the end-to-end generation view supplied the auxiliary reasoning trace. Because the development configurations were each evaluated with a single seed, small differences among them should be interpreted cautiously.

Our error analysis suggests that argument-value prediction remains the main bottleneck, particularly for free-text spans and list-valued fields, where span-boundary mismatches, semantic substitutions, and hallucinated values are common. Performance also varies across dialects, although results for dialects with few examples should be interpreted cautiously.

8 Acknowledgments

This research was supported by The VNUHCM University of Information Technology's Scientific Research Support Fund.

We thank Tuwaiq Academy and the AISA-ArabicFC organizers for providing the dataset, the baseline model, and the evaluation framework for this shared task.

9 Declaration on Generative AI Use

ChatGPT and Codex were used to assist with rewriting, language refinement, error correction, and code-related tasks. All AI-assisted content was reviewed and revised by the authors, who take full responsibility for the final manuscript.

References

Asım Ersoy, Enes Altinisik, Kareem Mohamed Darwish, and Husrev Taha Sencar. 2025. Tool calling for arabic llms: Data strategies and instruction tuning. In *Proceedings of The Third Arabic Natural Language Processing Conference*, pages 347–358.

Konstantin Kubrak, Ahmed El-Moselhy, Ammar Alsulami, Remaz Altuwaim, Hassan Ismail Fawaz, and Faisal Alsaby. 2025. Arabic prompts with english tools: A benchmark. In *2025 IEEE International Conference on Big Data (BigData)*, pages 5452–5461. IEEE.

Mayank Kulkarni, Vittorio Mazzia, Judith Gaspers, Chris Hench, Jack FitzGerald, and AGI Amazon. 2025. Massive-agents: A benchmark for multilingual function-calling in 52 languages. In *EMNLP (Findings)*, pages 20193–20215.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 3102–3116.

Zheng Luo, T Pranav Kutralingam, Ogochukwu N Okoani, Wanpeng Xu, Hua Wei, and Xiyang Hu. 2026. Lost in execution: On the multilingual robustness of tool calling in large language models. In

Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 44059–44077.

Omer Nacar, Mohammed Al Khalifa, and Saeed Alzaharani. 2026a. AISA-ArabicFC: The First Shared Task on Arabic Function Calling for Agentic AI Systems. In *Proceedings of The Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.

Omer Nacar, Deema Alquffari, Saleh Alsharideh, Adeem AlOtaibi, Abdulaziz Alabdulkarim, Leen Alhazmi, Nada Alomar, Wareef Alzubaidi, Nada Alsultan, Ahmed Alrabghi, et al. 2026b. From language to action in arabic: Reliable structured tool calling via data-centric fine-tuning. *arXiv preprint arXiv:2603.16901*.

Omer Nacar, A. Deema, and A. Mohammed. 2026c. AISA: A unified architecture for agentic ai systems.

Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. 2025. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2024. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, volume 2024, pages 9695–9717.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551.